1. Selection Sort

Basic idea

Selection sort repeatedly finds the **smallest element** from the unsorted part and puts it at the beginning.

Algorithm

SelectionSort(A, n)

```
1. for i = 0 to n-2
2.     min = i
3.     for j = i+1 to n-1
4.         if A[j] < A[min]
5.             min = j
6.     swap A[i] and A[min]
```

```
#include <stdio.h>

int main()
{
   int arr[] = {64, 25, 12, 22, 11};
   int n = 5;
   int i, j, min, temp;

   // Selection Sort
   for (i = 0; i < n - 1; i++)
   {
      min = i;

      for (j = i + 1; j < n; j++)
      {
         if (arr[j] < arr[min])
         {
            min = j;
         }
      }

      temp = arr[i];
      arr[i] = arr[min];
      arr[min] = temp;
   }
```

```
    // Print sorted array
    printf("Sorted array: ");

    for (i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

C program

```
#include <stdio.h>

void selectionSort(int arr[], int n)
{
  int i, j, min, temp;

  for (i = 0; i < n - 1; i++)
  {
      min = i;

      for (j = i + 1; j < n; j++)
      {
          if (arr[j] < arr[min])
          {
              min = j;
          }
      }

      temp = arr[i];
      arr[i] = arr[min];
      arr[min] = temp;
  }
}

int main()
{
  int arr[] = {64, 25, 12, 22, 11};
  int n = 5;
```

```
    selectionSort(arr, n);

    printf("Sorted array: ");

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

Complexity

Case	Time
Best	**O(n²)**
Average	**O(n²)**
Worst	**O(n²)**
Space	**O(1)**

Why O(1) space?
Because selection sort sorts the array **in-place**. It only needs a few extra variables such as `i`, `j`, `min`, and `temp`.

1. Selection Sort — Count Comparisons and Swaps

```
#include <stdio.h>

int main()
{
  int arr[] = {64, 25, 12, 22, 11, 90};
  int n = 6;

  int i, j, min, temp;
  int comparisons = 0;
  int swaps = 0;

  for (i = 0; i < n - 1; i++)
  {
     min = i;

     for (j = i + 1; j < n; j++)
     {
        comparisons++;

        if (arr[j] < arr[min])
        {
           min = j;
        }
     }

     if (min != i)
     {
        temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;

        swaps++;
     }
  }

  printf("Sorted array: ");

  for (i = 0; i < n; i++)
  {
     printf("%d ", arr[i]);
  }
```

```
    printf("\nTotal comparisons: %d", comparisons);
    printf("\nTotal swaps: %d", swaps);

    return 0;
}
```

Output

```
Sorted array: 11 12 22 25 64 90
Total comparisons: 15
Total swaps: 4
```

Why 15 comparisons?

For 6 elements:

5 + 4 + 3 + 2 + 1 = 15

2. Selection Sort for Strings

For strings in C, you cannot use:

str1 < str2

Instead, use `strcmp()` from `<string.h>`.

```
#include <stdio.h>
#include <string.h>

int main()
{
    char arr[5][20] = {
        "Banana",
        "Apple",
        "Mango",
        "Orange",
        "Grapes"
    };

    int n = 5;
    int i, j, min;
    char temp[20];
```

```
    for (i = 0; i < n - 1; i++)
    {
        min = i;

        for (j = i + 1; j < n; j++)
        {
            if (strcmp(arr[j], arr[min]) < 0)
            {
                min = j;
            }
        }

        strcpy(temp, arr[i]);
        strcpy(arr[i], arr[min]);
        strcpy(arr[min], temp);
    }

    printf("Sorted strings:\n");

    for (i = 0; i < n; i++)
    {
        printf("%s\n", arr[i]);
    }

    return 0;
}
```

Output

Sorted strings:
Apple
Banana
Grapes
Mango
Orange

Important

For string Selection Sort:

- `strcmp()` → compares two strings
- `strcpy()` → copies one string to another
- `char temp[20]` → temporarily stores a string during swapping

The sorting logic is still the **same Selection Sort** you used for integers.